Instituto Tecnológico y de Estudios Superiores de Monterrey

Campus Guadalajara

Tarea 1 : Optimización de virtualización en React

Planeación de sistemas de software (Gpo 102)

Team 26

José Yael Varela García - A01645324

Danilo Paolo Tato Velázquez - A01644630

Sebastián Alejandro Soria Piñuela - A01645849

Sebastian Certuche - A01644942

February 28, 2026

Virtual DOM (VDOM)

El Virtual DOM es una representación ligera en memoria (un objeto JavaScript) del DOM real que React mantiene sincronizada.

¿Cómo funciona?

1. React crea un árbol de VDOM con la estructura de la interfaz (UI) y lo refleja en el DOM real.

2. Cuando ocurre un cambio en el estado o las propiedades de un componente, React genera un nuevo árbol de VDOM completo que representa cómo debería verse la UI post-cambio.

3. React compara el nuevo VDOM con la versión que tenía guardada del render anterior. Esta comparación (llamada Diffing) es nodo a nodo y se basa en una lógica de comparación descrita de la siguiente manera:

- Tipo : Si un elemento cambia de tipo (ej. <h1> a <title>) React marca ese nodo y a sus hijos para ser destruidos y creados de nuevo pero aplicando el cambio deseado.

- Atributos y estilos : Si el tipo es igual, React compara las propiedades, si identifica un cambio, anotará que debe cambiar ese atributo o estilo.

- Contenido : Compara el texto. Si el texto cambió React nuevamente lo detecta y marca para actualizarlo.

Listas: Si se tiene una lista, React utiliza Keys para que cada nodo tenga un ID único. Con ello, si un elemento cambia de posición, React lo detecte como la misma instancia, solo en una posición diferente, y con ello pueda marcarlo para reordanarlo en el DOM real, sin necesidad de destruir y recrear el nodo, haciendo más eficiente el proceso.

4. El último paso, llamado de Reconciliación, es el proceso donde React toma todas las diferencias encontradas en el paso anterior, el diffing, y aplica solo los cambios necesarios en el DOM real, evitando una reconstrucción total del DOM.

Relación con React.memo y map()

`React.memo` es una **función de orden superior (HOC)** que permite memorizar componentes funcionales en React con el objetivo de **evitar renderizados innecesarios**. Su uso está orientado a la optimización del rendimiento, especialmente en aplicaciones con componentes que se renderizan con frecuencia o listas grandes.

Cuando un componente está envuelto con React.memo, React realiza una comparación superficial (shallow comparison) entre las props actuales y las props del render anterior. Si

React detecta que las props no han cambiado, omite completamente el render del componente. En cambio, si alguna prop es diferente, el componente se vuelve a renderizar de forma normal.

Es importante diferenciar entre render y actualización del DOM. El render consiste en ejecutar la función del componente para generar un nuevo árbol de Virtual DOM, mientras que la actualización del DOM real implica modificar los nodos del navegador, lo cual es una operación más costosa. React.memo actúa antes del Virtual DOM, evitando que el componente genere un nuevo subárbol virtual cuando no es necesario.

A nivel del flujo interno de React, el comportamiento es el siguiente:

- El componente padre se renderiza debido a un cambio de estado.
- React evalúa los componentes hijos memorizados con React.memo.
- Si las props del hijo son idénticas a las anteriores, React cancela su render.
- No se genera un nuevo Virtual DOM para ese componente.
- No se realiza diffing ni reconciliación para ese subárbol.
- El DOM real permanece sin cambios.

Esto significa que React.memo no modifica directamente el DOM, sino que reduce el trabajo previo que React tendría que hacer sobre el Virtual DOM y, como consecuencia, disminuye las operaciones innecesarias sobre el DOM real.

Sin embargo, para que React.memo sea efectivo, es fundamental que las props sean referencialmente estables. En JavaScript, objetos, arreglos y funciones se comparan por referencia. Si una función se redefine en cada render del componente padre, React la considerará una prop distinta, aunque su lógica sea la misma. En ese caso, React.memo perdería su efecto.

Aquí es donde entra useCallback. Al envolver funciones como deleteItem o likeItem con useCallback, se garantiza que su referencia se mantenga estable entre renders, siempre que sus dependencias no cambien. Esto permite que React.memo confirme que las props no han cambiado y evite renderizados innecesarios en los componentes hijos.

En conjunto, React.memo y useCallback forman una estrategia de optimización sólida: useCallback mantiene estables las referencias de las funciones que se pasan como props, y React.memo utiliza esa estabilidad para omitir renders completos. El resultado es un menor número de nodos generados en el Virtual DOM, menos comparaciones durante el proceso de diffing y una reducción significativa de actualizaciones innecesarias en el DOM real.

Uso correcto de map() y key en listas dinámicas

En React, cuando se renderiza una lista utilizando map(), es obligatorio asignar una propiedad key a cada elemento generado. Esta key permite a React identificar de manera única cada componente dentro de la lista durante el proceso de reconciliación del Virtual

DOM, permitiendo así mantener su identidad aunque cambie de posición y re-renderizar únicamente el elemento que cambió.

En esta implementación, la lista es dinámica, ya que permite:

- Agregar elementos.
- Eliminar elementos.
- Invertir el orden de la lista.
- Modificar propiedades internas, como el contador de likes.

¿Qué pasaría si se usan incorrectamente las keys?

Reutilización incorrecta de componentes

React identifica los elementos por su key. Si la key cambia cuando el orden cambia, React puede reutilizar un componente equivocado, asignándole datos que no le corresponden.

Estados mezclados

Si un componente tiene estado interno, al cambiar el orden de la lista el estado puede moverse a otro elemento visual, generando comportamientos incorrectos.

Bugs visuales

- Likes que parecen cambiar de elemento.
- Inputs que conservan valores incorrectos.
- Componentes que se actualizan cuando no deberían.

Renderizados innecesarios

Cuando las keys no son estables, React no puede optimizar correctamente el renderizado y puede desmontar y montar componentes completos en lugar de simplemente actualizarlos.

Pongamos esto en práctica (Componente ListSection)

```javascript
export default function ListSection() {

  const [items, setItems] = useState([]);
```

```
  const [text, setText] = useState("");

  const addItem : arrow Function
  const reverseList : arrow Function
  const deleteItem : useCallback <- Function
  const likeItem : useCallback <- Function

  return (
    <div>
      <input
        value={text}
        onChange={(e) => setText(e.target.value)}
      />
      <button onClick={addItem}>Add</button>
      <button onClick={reverseList}>Reverse</button>
      <ul>
        {items.map(item => (
          <ListItem
            key={item.id}
            item={item}
            deleteItem={deleteItem}
            likeItem={likeItem}
          />
        ))}
      </ul>
    </div>
  );
}
```

La implementación de esta lista dinámica permite observar de forma concreta cómo optimizar el uso del DOM en React. Cada vez que cambia el estado de ‘items’, React genera un nuevo árbol de Virtual DOM y lo compara con el anterior. A partir de esa comparación, identifica únicamente las diferencias y aplica solo los cambios necesarios en el DOM real, evitando reconstrucciones completas.

En este proceso, la propiedad key={item.id} es fundamental. La key representa la identidad única de cada componente dentro del árbol virtual. Al generar el identificador una sola vez con crypto.randomUUID() al momento de crear el elemento, se garantiza que cada nodo mantenga una identidad estable. Esto permite que, al invertir la lista, React detecte que los elementos son los mismos pero en distinto orden, y simplemente los reordene en el DOM real sin destruirlos ni recrearlos. Si en este caso se utilizara el índice como key, el cambio de posiciones alteraría esas claves y React interpretaría que los componentes son distintos, provocando operaciones innecesarias.

Por otra parte, el uso de ‘useCallback’ en ‘deleteItem’ y ‘likeItem’ es relevante porque estas funciones se pasan como props a cada ‘ListItem’. En JavaScript, las funciones se comparan por referencia; si se recrearan en cada render, React detectaría un cambio en las props y podría volver a renderizar los hijos innecesariamente. Al mantener estable la referencia mediante ‘useCallback’, se evita que esos cambios superficiales rompan la optimización. Es decir, ‘useCallback’ embona muy bien con la función ‘memo’.

Tabla de actividades

Integrante	Actividad
Danilo	Explicación de ListSection y la importancia del uso de useCallback. Código de ListSection.
Sebastian Soria	Uso correcto de map() y key en listas dinámicas y, ¿que pasaría si se usan incorrectamente las keys? Código de CounterSection.
José Yael	Explicación del VDOM en React. Código de CounterSection.
Sebastian Certuche	Explicación de la función memo en React.